

Gipfelsturm - Project Report*

Robin Gibson

Andrej Scheuer

Lorenzo Celli

Oliver Saravanja

1. Project Overview

We investigate throughput optimization for GPT-style model training on the CSCS Alps supercomputer (using NVIDIA GH200 Grace-Hopper nodes) using Megatron-LM, pursuing two independent approaches: FP8 mixed-precision training and kernel-level optimizations via Liger kernels and `torch.compile`. Our main findings are: FP8 delivers up to $\sim 41\%$ throughput improvement over BF16 at 8B parameters on a single node, translating into lower validation loss at a fixed compute budget, but provides no benefit below ~ 3 B parameters where quantization overhead outweighs the gains; Liger kernel drop-in replacements regress throughput by 22% at 760M due to tensor layout incompatibilities with Megatron-LM’s existing JIT-fused kernels, and are neutral at 8B; `torch.compile` with default mode yields modest gains (~ 1 – 2%) at 3B and 8B parameters but is detrimental at smaller scales. Code, patches, and results are available at <https://github.com/RobGibson/lisaie-ss26-gipfelsturm>.

2. Motivation

Training large language models is extremely resource- and compute-intensive, making it a costly task. Because compute clusters are leased by GPU-hour, a $2\times$ improvement in throughput translates directly into a $2\times$ reduction in cost, and maximizing throughput is therefore a primary systems-level objective in large-scale training. The simplest way to absorb the growing compute demand is to scale out to more hardware, but hardware budgets are finite; the complementary approach is to extract more useful work from each GPU through algorithmic and numerical improvements. We investigate the latter on the Alps supercomputer, training GPT-style models with Megatron-LM [9] on NVIDIA GH200 Grace-Hopper nodes. Alps is well suited to this investigation because each GH200 exposes fourth-generation Tensor Cores with native FP8 support: at dense throughput, FP8 reaches roughly 1,979 TFLOP/s against 989 TFLOP/s for BF16 on the same silicon, a $2\times$ theoretical headroom over the current BF16 default, enabling accelerat-

ing for deep learning training beyond the 16-bit formats [6]. Whether the theoretical $2\times$ translates into a real throughput gain in Megatron-LM, across which model scales, and at what cost to convergence, is the empirical question this work addresses.

Alongside FP8 we investigate kernel-level optimizations as an alternative approach to increasing throughput. Every transformer training step dispatches hundreds of individual CUDA kernels; each launch incurs Python dispatch overhead and forces intermediate results to be stored in memory before the next operator can consume them. This reduces the effective arithmetic intensity. We explore two approaches to close this gap: The first approach is to use *Liger Kernels* [5], which replaces specific element-wise operators with hand-written Triton kernels. The second approach is to use `torch.compile` [3], to apply TorchDynamo graph capture and Inductor code generation across the full model, enabling cross-boundary fusion that per-function `jit.script` cannot reach.

3. Method and Contribution

We pursue Challenge 2 along two complementary axes: FP8 mixed-precision training (section 3.1) and kernel-level optimizations (sections 3.2). Both target tokens-per-GPU-second on GH200 without sacrificing convergence. The two contributions are independent in implementation but share the same training stack (Megatron-LM `core_v0.16.1` with TransformerEngine[7], and the Nemotron-ClimbMix dataset [4]), and are evaluated on the same model suite (125M to 8B parameters).

3.1. FP8 mixed-precision training

Motivation and central claim. FP8 offers a $2\times$ theoretical peak-throughput advantage over BF16 on Hopper Tensor Cores, but realizing this advantage in practice is non-trivial. FP8’s narrow dynamic range requires per-tensor scaling factors, the storage and update of which add fixed overhead per kernel. Extra FP8 copies of weights/activations (the quantized versions, 1 byte each), the scaling-factor and amax-history metadata increase memory pressure relative to BF16, making it less feasible to use in certain scenarios. To do an FP8 general matrix multiplication (GEMM), TransformerEngine has to

*This work was done as part of the Large-Scale AI Engineering course of ETH Zurich.

(a) cast the BF16/FP32 tensors to FP8, (b) compute or apply amax/scale factors, and (c) often cast the output back. At large problem sizes, the GEMM itself dominates and these casts amortize away. At small problem sizes, the GEMM is so cheap that the casting and scaling overhead is a relatively larger fraction of step time. We hypothesize, and confirm empirically, a *model-size-dependent crossover*: below roughly 3B parameters, these fixed costs are relatively too big and FP8 provides no benefit (or a slight loss) over BF16. Above the crossover, FP8 clearly wins. Our contribution is a systematic characterization of this crossover on GH200, decomposed into the following experiments.

Recipe and format selection. TransformerEngine exposes several FP8 scaling strategies, of which we evaluate the three relevant for Hopper: *delayed scaling* (per-tensor scale computed from an amax-history window of past iterations), *current/tensorwise scaling* (per-tensor scale computed within the same step), and *blockwise scaling* (per-block scale, the strategy used by DeepSeek-V3). Each strategy can be combined with either the pure four exponent and 3 mantissa bits (E4M3) format or the *hybrid* format (E4M3 in the forward pass for weights and activations, E5M2 in the backward pass for gradients, where the wider exponent better tolerates gradient outliers). We performed sweeps on 760M and 8B model sizes. Results are reported in section 4.1.

Throughput across model scales (H_{F1}). To test the crossover hypothesis, we run BF16 and FP8 hybrid-delayed on the full model suite (125M, 350M, 760M, 1.5B, 3B, 8B), keeping global batch size, sequence length, and parallelism degree constant (GBS=256, seq_len=4096, TP=PP=1). The single per-precision throughput number reported at each model size is the median of 45 measured steps (50 total minus 5 warmup steps to exclude JIT and Triton-cache transients). Results are reported in section 4.1.

Configuration unlock at 8B (H_{F2}). At 8B the picture is more nuanced because memory pressure constrains the configuration space. With BF16 and TP=1, MBS=2 fits comfortably. But with FP8 under the same parallelism, MBS=2 runs out of memory due to the additional FP8 memory overhead. The natural workarounds are (i) reduce micro-batch size to MBS=1, trading per-step GEMM efficiency for memory headroom, or (ii) introduce intra-node tensor parallelism (TP=2, MBS=2), trading memory for additional all-reduce communication. We ran the full $2 \times 2 \times 2$ matrix (BF16/FP8) $\times$ (TP=1/TP=2) $\times$ (MBS=1/MBS=2) plus the OOM cell to test whether FP8’s throughput advantage survives these configuration adjustments. Results are in section 4.1.

Convergence at fixed budget (H_{F3}). A throughput improvement is only useful if model quality is preserved, and on the Alps cluster compute is the binding constraint. We therefore evaluate the combination as a practitioner would: at fixed wall-clock budget. We train the 8B model under both precisions for an identical number of GPU-hours (one node, 4 GPUs) and log validation loss against both training steps and elapsed time. The per-step comparison isolates the question “does FP8 quantization degrade learning per step?”. The per-time comparison answers the question “does the throughput gain translate into a lower loss at fixed budget?”. Results are in section 4.1.

Strong scaling (H_{F4}). Finally we perform a strong-scaling experiment comparing FP8 to BF16. As FP8 reduces compute time per step but does not reduce gradient communication volume, the FP8 throughput advantage should shrink as the data-parallel degree grows, because communication occupies a growing share of step time—especially once it must cross the slower inter-node Sling-shot interconnect (~ 93 GB/s measured, versus ~ 340 GB/s intra-node NVLink)—while the accelerated compute portion shrinks in relative weight. We test this by running the 8B model in both precisions on 1, 2, 4, and 8 nodes (4–32 GPUs) with TP=PP=1 and GBS held constant at 256. Results are in section 4.1.

3.2. Kernel-level optimizations

Motivation and central claim. Megatron-LM already applies per-function kernel fusion via `torch.jit.script (@jit_fuser)` to individual activation functions, but we believe that two distinct inefficiencies remain. First, PyTorch’s JIT-compiled CUDA kernels may be suboptimal for element-wise operations compared to hand-tuned Triton alternatives. Second, operators across module boundaries remain unfused entirely (e.g., the layer-normalization output feeding directly into a linear projection). We target each with a dedicated approach. The *Liger kernels* library [5] offers, among others, a drop in replacement for SwiGLU, GeGLU, and RMSNorm implementations with hand-written Triton kernels. Using these, we aim to achieve better memory access efficiency within these operators. To address the module boundary issue, we apply `torch.compile` [3] to the whole model. This approach compiles the full model graph via TorchDynamo and Inductor. Dynamo inlines through the existing `@jit_fuser` wrappers and compiles the resulting larger graph, enabling the cross-boundary fusion that per-function `jit.script` cannot reach. Results for the Liger kernels can be found in 4.2 and for `torch.compile` in 4.3.

3.2.1. Liger kernels

Implementation The Liger Kernel library supports general and model-specific automatic patching of models [5], but at the time of writing this does not extend to Megatron-LM [1]. We therefore do the integration manually via three patches to Megatron’s fusion layer. The `SwiGLUFunction` and `GeGLUFunction` custom autograd classes have their forward and backward passes replaced with `liger_kernel.ops.swiglu` and `liger_kernel.ops.geglu` respectively, and the existing `@jit_fuser` implementations are removed. `RMSNorm` is replaced with `LigerRMSNorm` from `liger_kernel.transformers`, bypassing PyTorch’s native `torch.nn.RMSNorm`. Because the Liger `SwiGLU` and `GeGLU` ops operate on split half-tensors and internally call `.view()`, which requires contiguous memory, `.contiguous()` calls are inserted around each `torch.chunk` output in both the forward and backward passes.

The Liger Kernel library is not part of the provided pre-built docker image used to run Megatron-LM. To address this we opted to install it as a site-package in the scratch partition and link it at run time with the `launch.sh` script.

Setup. All Liger kernel experiments are run on 4 Alps nodes (16 GH200 GPUs in total) in BF16 precision with `TP=1`, `PP=1`, `GBS=256`, `MBS=4`, and a sequence length of 4096 tokens. We evaluate two model sizes: 760M and 8B parameters. The 760M model is chosen because its smaller GEMM footprint means element-wise activations and normalization represent a comparatively larger share of time per step, making it a favourable setting for operator-level kernel improvements. The 8B model represents the GEMM-dominated regime where those operators are expected to be negligible. Each run executes 60 iterations. The first 10 are used for warmup and are therefore excluded from the analysis. Throughput is measured over the remaining 50 steps. The reported figure 5 is the median tokens-per-GPU-second over steady-state steps, which is robust to the occasional garbage-collection and data-loading spikes visible in the per-step traces.

Hypotheses. We test two hypotheses.

- **H_{L1}**: at 760M parameters, activation and normalization kernels are a non-trivial fraction of computation time per step. Liger’s custom Triton kernels should yield a measurable throughput improvement over the `@jit_fuser` baseline.
- **H_{L2}**: at 8B parameters, GEMMs dominate compute. Therefore the effect of replacing these operators with Liger Triton kernels should be negligible.

3.2.2. torch.compile

Implementation For this implementation, we added the flags `--torchcompile` and `--torch-compile-mode` to Megatron’s argument parser. When enabled, each pipeline-stage model chunk is wrapped with `torch.compile`, and the argument passed to `--torch-compile-mode` is forwarded to `torch.compile` as the mode argument. The `torch.compile` is inserted after `Float16Module` wrapping but before DDP wrapping, so that inter-stage communication calls are not captured in the compiled graph. `disable_jit_fuser()` is called immediately after to prevent redundant per-function compilations, since Dynamo already inlines through the `@jit_fuser` decorators when tracing the larger graph. Three compilation modes are supported and have been tested to work:

1. `default`: standard fusion, no CUDA graphs. Should be a good balance between performance and overhead.
2. `reduce-overhead`: CUDA graph capture to amortize kernel-launch overhead, which reduces the overhead of python with CUDA graphs, useful for small batches.
3. `max-autotune-no-cudagraphs`: Kernel autotuning that leverages Triton or template based matrix multiplications on supported devices and Triton based convolutions on GPU. Does not use CUDA graphs.

The above variants and short descriptions are taken from the official PyTorch documentation [8]. We also tried `max-autotune`, i.e. autotuning with CUDA graphs. However, we encountered runtime issues where the tensor output of `CUDAGraphs` was being overwritten by subsequent iterations. This also affected the `reduce-overhead` variant. We attempted to pinpoint and address the issue and managed to solve it for the `reduce-overhead` case, but not for `max-autotune`. As an alternative, we opted to use `max-autotune-no-cudagraphs` instead. To enable the `reduce-overhead` compilation an additional patch was developed, which disables Dynamo tracing in `TEDotProductAttention`. Disabling Dynamo tracing creates a graph break at each attention call so that fused attention runs in eager mode while the surrounding linear projections remain compiled in the outer CUDA graph.

Setup. All experiments are run on 4 Alps nodes (16 GH200 GPUs), in BF16 with `TP=1`, `PP=1`, `GBS=256`, `seq_len=4096`. We compare the `@jit_fuser` baseline against the three working compilation modes: `default`, `reduce-overhead`, and `max-autotune-no-cudagraphs` (labelled *Max Autotune* in the figures). Each run executes 100 measured iterations after 10 warmup iterations; the per-step throughput distributions are shown as violin plots in figures 6–8.

Hypothesis. H_{T1} : whole-model `torch.compile` enables cross-boundary fusion unreachable by `per-function jit.script`, and should therefore improve throughput over the baseline, with the largest gains expected from the autotuning mode.

4. Evaluation

We organize the FP8 results around the four hypotheses laid out in section 3.1: (H_{F1}) a model-size crossover above which FP8 outperforms BF16; (H_{F2}) FP8’s throughput advantage at 8B survives the configuration changes its memory footprint forces; (H_{F3}) the throughput gain translates into lower validation loss at fixed compute budget; and (H_{F4}) the gain shrinks as the data-parallel degree grows. All FP8 results use the hybrid-delayed recipe unless explicitly stated otherwise; all reported per-step quantities are medians over the last 45 of 50 measured steps, discarding the first five to exclude JIT compilation and Triton cache warm-up.

4.1. FP8 throughput

Recipe and format selection. We swept all viable (format, recipe) combinations at 760M and a subset at 8B. At 760M the differences between FP8 recipes are small compared to the FP8-vs-BF16 gap itself, with hybrid-blockwise and e4m3-delayed slightly ahead. These ordering effects do not carry over to 8B: there, hybrid-delayed is the strongest configuration, consistent with the TransformerEngine library default [2]. We adopt hybrid-delayed throughout the remainder of this work. Two observations are worth noting. First, the ranking of recipes is not stable across model scales, which is consistent with our central claim that the FP8 cost-benefit balance is itself scale-dependent. Second, no FP8 recipe rescues the 760M crossover: at this scale, BF16 remains competitive regardless of which FP8 strategy is chosen, so recipe selection is not the explanation for the small-model loss reported below.

Throughput across model scales (H_{F1}). As seen in figure 1, below approximately 3B parameters BF16 matches or slightly exceeds FP8; above 3B, FP8 wins by a margin that grows with model size, reaching $\sim 41\%$ at 8B (section 4.1). This is direct evidence for the model-size crossover predicted in section 3.1: at small scale, the fixed costs of FP8 (casting, scaling-factor maintenance, reduced MBS headroom) are comparable in magnitude to the GEMMs themselves and dominate any arithmetic benefit; above the crossover, GEMM cost grows faster than the FP8 overhead and the $2\times$ peak-throughput headroom begins to materialize.

Configuration matrix at 8B (H_{F2}). Figure 2 shows the throughput distribution of all six 8B configurations. Three

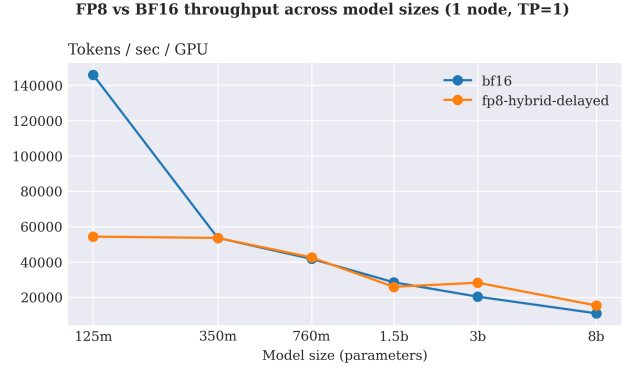


Figure 1. Tokens per second per GPU over various sizes (BF16 vs FP8, TP=PP=1, GBS=256).

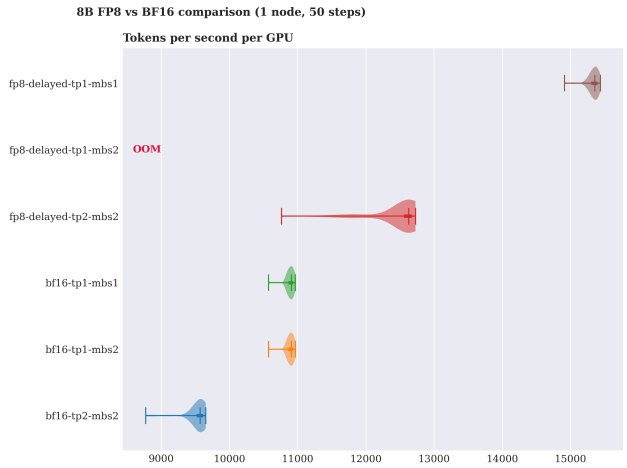


Figure 2. Configuration Insights on 8B model (BF16 vs FP8).

findings stand out. *First*, the fastest FP8 configuration (TP=1, MBS=1) reaches $\sim 15,400$ tokens/sec/GPU against the fastest BF16 configuration’s $\sim 10,900$, a $\sim 41\%$ advantage. *Second*, the configuration FP8 is forced into by its memory footprint is *not* the configuration BF16 prefers: BF16 fits TP=1/MBS=2 comfortably (and indeed prefers it), but FP8 OOMs in that cell and must drop to MBS=1 or pay a TP=2 communication cost. Even from this handicapped starting point, FP8 with TP=2/MBS=2 at $\sim 12,600$ tokens/sec/GPU still beats every BF16 configuration. The FP8 advantage is therefore robust to the configuration adjustments its memory footprint forces. *Third*, BF16 with TP=1 is essentially indifferent to micro-batch size (MBS=1 and MBS=2 both reach $\sim 10,900$), suggesting that at 8B with BF16 the workload is no longer GEMM-size-bound and the conventional “larger MBS, better throughput” heuristic does not apply.

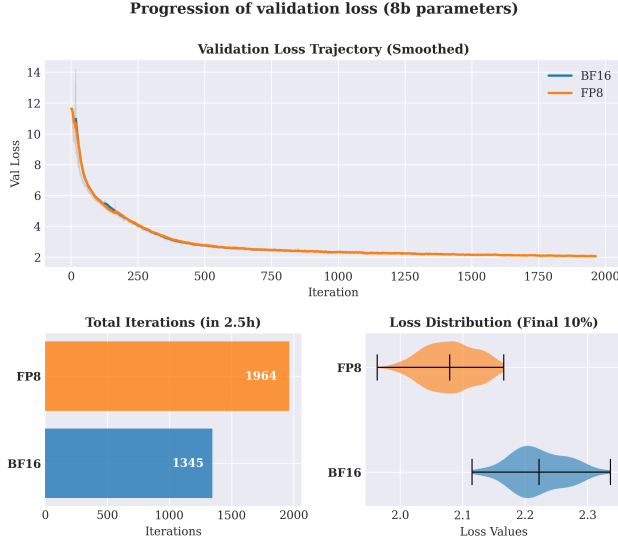


Figure 3. Improved convergence given fixed training duration (FP8 vs. BF16)

Convergence at fixed budget (H_{F3}). Figure 3 shows the 8B comparison under a fixed 2.5 hour, single-node budget. The smoothed validation-loss trajectories against iteration are visually indistinguishable: FP8 quantization does not degrade per-step learning quality at this scale, confirming the secondary claim of H_{F3} . In the same wall-clock window FP8 completes 1,964 iterations to BF16’s 1,345, a ratio of $\sim 1.46\times$ that is consistent with the $\sim 1.41\times$ throughput advantage measured at the optimal configuration (H_{F2}). Because per-iteration loss is matched and FP8 takes more iterations, FP8 ends the budget at a lower validation loss: the distribution of the final 10% of iterations is centered at ~ 2.08 LM loss for FP8 versus ~ 2.22 for BF16, with the two distributions clearly separated. The throughput gain therefore converts directly into model quality at fixed compute, which is the practitioner’s relevant metric.

Strong scaling (H_{F4}). Figure 4 reports tokens/sec/GPU for the 8B model across 4, 8, 16, and 32 GPUs (1, 2, 4, 8 nodes) at $TP=PP=1$ and $GBS=256$. Both precisions show the expected per-GPU degradation as the data-parallel degree grows, but the slopes differ sharply. BF16 declines from $\sim 10,900$ at 4 GPUs to $\sim 10,400$ at 32 GPUs ($\sim 95\%$ parallel efficiency relative to its 1-node value). FP8 declines from $\sim 15,400$ to $\sim 11,400$ over the same range ($\sim 74\%$ efficiency), with the largest single drop occurring between 4 and 8 GPUs — precisely the transition from intra-node NVLink to inter-node Slingshot. As a consequence, the FP8-over-BF16 advantage contracts from $\sim 41\%$ on a single node to $\sim 10\%$ at eight nodes. This is the Amdahl signature predicted in section 3.1: FP8 accelerates only the

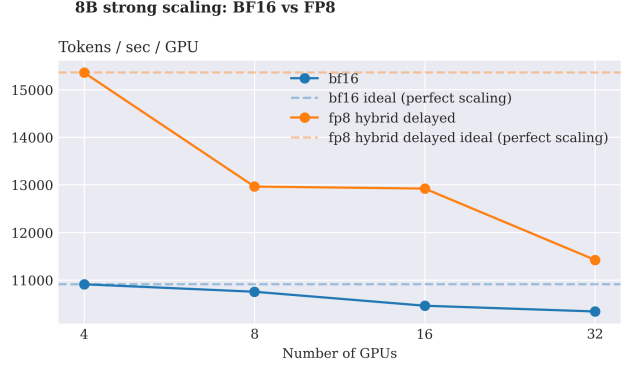


Figure 4. Scalability BF16 vs FP8 on 8B model.

compute portion of step time, so as communication occupies a growing share, FP8’s accelerable fraction shrinks and the achievable speedup falls toward unity. The same logic that produces the model-size crossover — FP8 needs the GEMMs to be the bottleneck — also produces this scaling behavior, and the largest FP8 gains will be obtained at the smallest data-parallel degree that still fits the model.

4.2. Liger kernel throughput

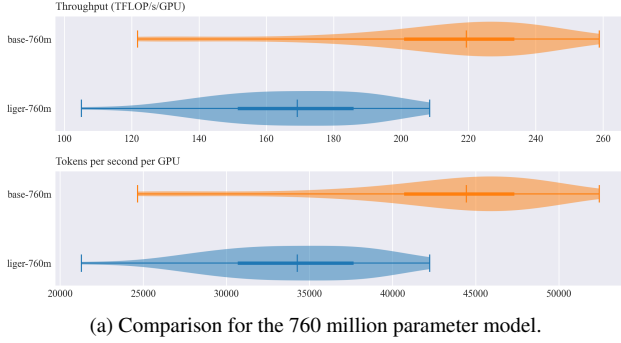
Table 1. Median steady-state throughput measured in token and TFLOP per second per GPU. The measurements were taken over 50 steps after 10 warmup steps, for the baseline and Liger-patched Megatron-LM on 4 nodes (16 GH200 GPUs), BF16, $GBS=256$, $seq_len=4096$. The best tok/s/GPU is marked in **bold**.

Model	Config	tok/s/GPU	TFLOP/s/GPU	Δ
760M	Base	44,805	221.2	—
760M	Liger	34,763	171.6	−22.4%
8B	Base	10,084	467.2	—
8B	Liger	10,161	470.8	+0.8%

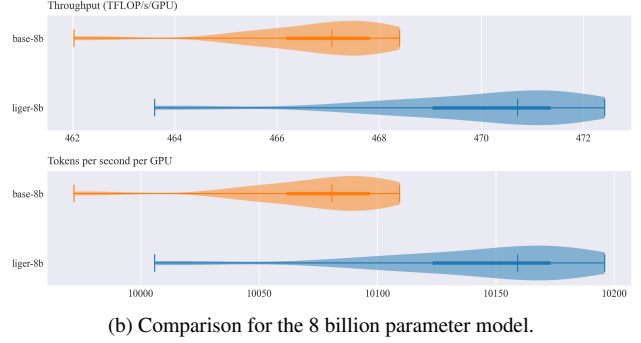
Results. Figure 5 shows per-step throughput for baseline and Liger runs at both scales, with table 1 listing the medians. At 760M, the Liger-patched model achieves a median of 34,763 tok/s/GPU against the baseline’s 44,805 tok/s/GPU. This is a **−22.4% regression**. We therefore have to reject the hypotheses H_{L1} . Liger’s Triton kernels are slower than the existing `@jit.fuser` implementation at this scale. At 8B, the medians are 10,161 vs 10,084 tok/s/GPU (+0.8%), well within run-to-run noise. H_{L2} is confirmed: operator-level kernel changes have no meaningful effect in the GEMM-dominated regime.

The measured TFLOP/s/GPU (see 1) support these observations and reinforce the decision to reject H_{L1} and confirm H_{L2} .

The 760M regression is explained by two factors iden-



(a) Comparison for the 760 million parameter model.



(b) Comparison for the 8 billion parameter model.

Figure 5. Throughput and tokens per second comparison of the baseline Megatron-LM (in blue) and Megatron-LM patched with Liger Kernel functions (in orange). All models were run on 4 nodes for 50 iterations with 10 warmup iterations.

tified during implementation (section 3.2.1). First, the `.contiguous()` calls inserted to satisfy Liger’s memory layout requirement introduce additional HBM copies that are absent in the `@jit_fuser` path. Second, the existing JIT-fused SwiGLU and GeGLU operate on the full unsplit $[B \times S, 2H]$ tensor — the most cache-efficient access pattern — whereas Liger receives two separately chunked halves of size $[B \times S, H]$, each with stride $2H$, resulting in strided HBM accesses that reduce effective memory bandwidth.

Resolving these overheads would require restructuring how Megatron-LM prepares tensors ahead of the gated activation functions, which would have required a more extensive refactor. We chose the Liger kernels because they looked like good drop-in replacements, and a basic integration was indeed straightforward. Achieving a performance-neutral or better integration, would have required the deeper tensor-layout refactor described above.

4.3. Torch.compile throughput

Results. Median throughputs are summarized in Table 2. The outcome is strongly mode-dependent and the best mode depends on scale, so H_{T1} holds only partially.

At 760M (figure 6) all compile modes are detrimental, considerably decreasing the throughput with respect to the baseline.

At 3B (figure 7) `default` yields the best results (~ 457) followed by `max-autotune-no-cudagraphs` (~ 452), both above the baseline (~ 448), while `reduce-overhead` regresses to ~ 440 .

At 8B (figure 8) the ordering changes again: `default` is the clear winner at ~ 493 versus the baseline’s ~ 488 , whereas `max-autotune-no-cudagraphs` (~ 481) and especially `reduce-overhead` (~ 467) both regress. It should be noted that in this case distributions seem to be bimodal, therefore the median is not a reliable comparison metric. Nevertheless, `default`’s entire distribution is shifted toward higher throughput relative to the baseline.

Two patterns emerge: `reduce-overhead` is never beneficial, suggesting CUDA-graph launch-overhead savings matter less as GEMMs grow. `max-autotune-no-cudagraphs` surpasses the baseline for a 3B model, but plain `default` fusion is sufficient and fastest for the 8B model as well. We therefore partially confirm H_{T1} : `torch.compile` can improve throughput over the `@jit_fuser` baseline, but only for larger scales.

Table 2. Median throughput (TFLOP/s/GPU) for the `@jit_fuser` baseline and the three `torch.compile` modes, across model scales. Measured over the steady-state iterations of each run (warmup excluded); BF16, TP=PP=1, GBS=256, seq_len=4096. Best per column in **bold**.

Config	760M	3B	8B
Baseline	139.9	448.1	487.8
Default	113.3	457.0	493.2
Reduce Overhead	113.8	440.3	467.1
Max Autotune	111.9	452.0	481.2

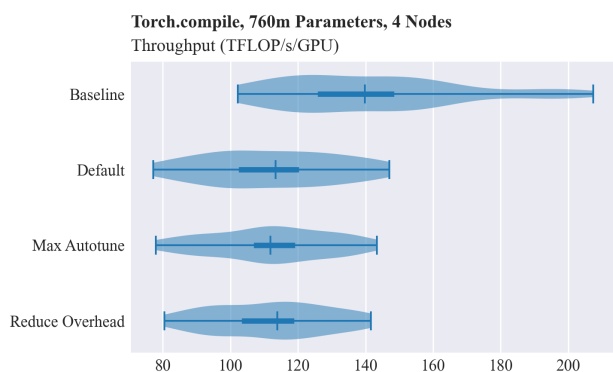


Figure 6. Per-step throughput (TFLOP/s/GPU) for the 760M model on 4 nodes (16 GH200 GPUs). The baseline surpasses all compile modes.

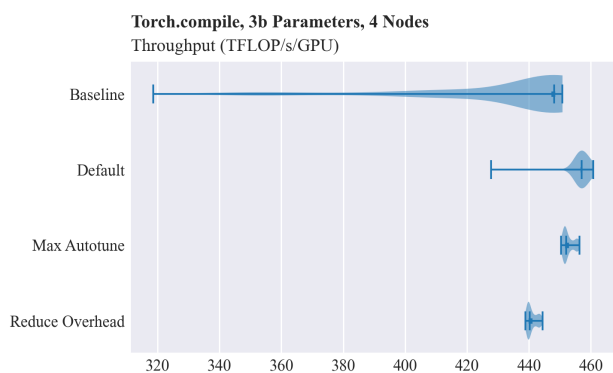


Figure 7. Per-step throughput (TFLOP/s/GPU) for the 3B model on 4 nodes (16 GH200 GPUs). Default and Max Autotune beat the baseline, Reduce Overhead regresses.

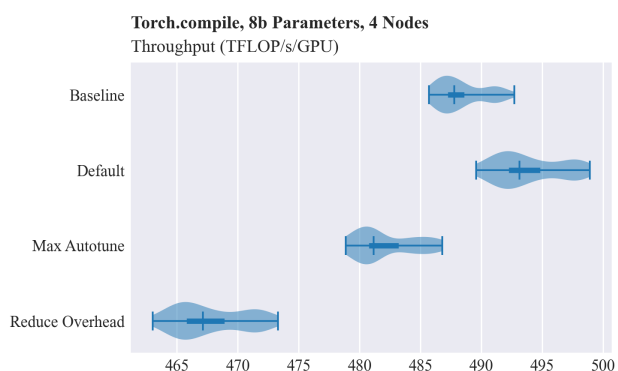


Figure 8. Per-step throughput (TFLOP/s/GPU) for the 8B model on 4 nodes (16 GH200 GPUs). Default is fastest, Max Autotune and Reduce Overhead both regress below the baseline.

References

- [1] Megatron support, 2025. Accessed: 2026-05-28. [3](#)
- [2] Transformer engine documentation, 2026. Accessed: 2026-05-29. [4](#)
- [3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Zach DeVito, Martin Raison, Nakul Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024. [1](#), [2](#)
- [4] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Celine Lin, Jan Kautz, and Pavlo Molchanov. Climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *arXiv preprint*, 2025. [1](#)
- [5] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Hamid Shojanazeri, Less Wright, Hanyu Zhao, Zhewei Yao, Yikang Shen, Jonah Shin, Ying Li, Guohao Chen, Tianhao Sun, Jason Liang, Mingqiang Huang, Wenpeng Yin, Maxim Naumov, and Dhruv Mahajan. Liger kernel: Efficient triton kernels for LLM training, 2024. [1](#), [2](#), [3](#)
- [6] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. Fp8 formats for deep learning, 2022. [1](#)
- [7] NVIDIA. Transformer engine. <https://github.com/NVIDIA/TransformerEngine>, 2024. [1](#)
- [8] PyTorch Contributors. `torch.compile` — PyTorch 2.12 documentation, 2025. Accessed: 2026-05-28. [3](#)
- [9] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. [1](#)